数据与算法

第8章 排序

北京科技大学
计算机与通信工程学院
杜　冰/皇甫伟

本节内容

- 排序的概念
- 基本排序方法
- 快速排序
- 归并排序
- 堆排序
- 线性复杂度排序

排序的概念

- 排序(Sorting)即是将杂乱无章的数据照一定规则顺次排列，计算机数据处理的主要操作之一 (>25%)
- 数据表(Datalist)：待排序数据元素的有限集合
- 关键字(Key)：数据元素有多个属性域，其中一个可以作为排序的依据
- 排序：根据关键字递增或递减的顺序，把数据元素依次排列起来，成为一组**关键字线性有序的元素**
  - n个元素的序列$\{R_{[0]}, R_{[1]}, \dots, R_{[n-1]}\}$，对应的关键字序列$\{K_{[0]}, K_{[1]}, \dots, K_{[n-1]}\}$
  - 确定某种排列$p[0], p[1], \dots, p[n-1]$，使得关键字满足

    非递减关系：$K_{[p[0]]} \le K_{[p[1]]} \le \cdots \le K_{[p[n-1]]}$

    非递增关系：$K_{[p[0]]} \ge K_{[p[1]]} \ge \cdots \ge K_{[p[n-1]]}$

排序的概念

- **逆序**：待排序序列中如果两个数据元素顺序不符合排序目标，称为一个逆序(倒置)；**反映初始序列和目标序列的差距**，可能决定**排序算法的效率**
- 排序算法的稳定性:元素序列中两个元素关键字相等，排序前后这两个元素的相对位置不变化，称**排序方法稳定**；否则称为**不稳定**；不同应用背景有不同要求
- 内排序/外排序：
  - 内排序：排序期间数据元素全部存放在内存中
  - 外排序：排序期间元素过多，不能同时存放在内存中，需要在内、外存之间移动元素

排序的概念

- **排序算法的性能评估**
  - 排序的时间开销可以用算法执行的**数据比较次数**、**数据移动次数**衡量
  - 时间开销（一般）：采用**平均情况**衡量
  - 时间开销：若受序列初始情况影响较大，需按照**最好、最坏情况分析**
  - 排序算法需要消耗的**额外的存储空间**也是衡量性能的重要特征，通常三种类型：
    - 除了小规模的堆栈或表，无需额外空间
    - 使用链表或指针，数组索引代表数据
    - 需要额外空间存储待排序元素序列副本
  - 寻找性能更好的排序算法是计算机科学重要课题

本节内容

- 排序的概念
- 基本排序方法
- 快速排序
- 归并排序
- 堆排序
- 线性复杂度排序

基本排序方法

- **比较和交换是排序的基本操作**

```
template <class ElemType>
inline void exch(ElemType &e1, ElemType &e2){
      ElemType tmp;
      tmp = e1;
      e1 = e2;
      e2 = tmp;
}


template <class ElemType>
inline void compExch(ElemType &e1, ElemType
&e2){
      if(e1>e2)  exch(e1, e2);
}
```

基本排序方法

- **冒泡排序(Bubble Sort)**
  - n个元素序列$\{R_{[0]}, R_{[1]}, \dots, R_{[n-1]}\}$待排序
  - 首先比较$R_{[n-1]}$和$R_{[n-2]}$的关键字，若逆序则交换这两个元素
  - 对$R_{[n-2]}$和$R_{[n-3]}$做同样处理，如此继续直到处理完$R_{[0]}$和$R_{[1]}$，完成一趟“冒泡”
  - 一趟冒泡可以将当前最小（or 最大）关键字移动到正确位置
  - 最多$n-1$趟冒泡即可完成排序

```
void bubble_sort(int *a, int left, int right){
    for(int i=left; i<right; i++){
        for(int j=right; j>i; j--){
            compExch(a[j-1], a[j]);          //逆序则交换
        }
    }
}
```

基本排序方法

	65	83	79	82	84	73	78	71	69	88	65	77	80	76	69
1	65	**65**	83	79	82	84	73	78	71	69	88	69	77	80	76
2	65	65	**69**	83	79	82	84	73	78	71	69	88	76	77	80
3	65	65	69	**69**	83	79	82	84	73	78	71	76	77	88	80
4	65	65	69	69	**71**	83	79	82	84	73	78	76	77	88	80
5	65	65	69	69	71	**73**	83	79	82	84	76	78	77	80	88
6	65	65	69	69	71	73	**76**	83	79	82	84	77	78	80	88
7	65	65	69	69	71	73	76	**77**	83	79	82	84	78	80	88
8	65	65	69	69	71	73	76	77	**78**	83	79	82	84	80	88
9	65	65	69	69	71	73	76	77	78	**79**	83	80	82	84	88
10	65	65	69	69	71	73	76	77	78	79	**80**	83	82	84	88
11	65	65	69	69	71	73	76	77	78	79	80	**82**	83	84	88
12	65	65	69	69	71	73	76	77	78	79	80	82	**83**	84	88
13	65	65	69	69	71	73	76	77	78	79	80	82	83	**84**	88
	65	65	69	69	71	73	76	77	78	79	80	82	83	84	88

基本排序方法

- **冒泡排序的性能**
  - 第i趟冒泡过程需要的比较次数为$n-i$，总的比较次数为：

$$\sum_{i=1}^{n-1}(n-i)=n(n-1)/2$$

  - 执行交换的次数在最坏的情况下也为$n(n-1)/2$
  - 冒泡排序是一种稳定的排序方法(等值元素相对位置不变)
- **冒泡排序的改进**
  - 某些序列无需完成所有的$n-1$趟冒泡即完成排序
  - **提前停止：**如果某趟冒泡过程中没有出现逆序和交换操作，即表明序列已经排序完成，后面的冒泡不用再进行
  - 最好(有序)情况下，比较n次，交换0次
  - 在平均和最快情况下，复杂度仍为$O(n^2)$

基本排序方法

- **插入排序(Insert Sort)**
  - 排序的每步将一个待排序元素，按照关键字大小，插入已经排序好的一组元素适当位置
  - 类似于抓牌过程中对牌进行排序的过程
  - 依据确定插入位置的方法不同，有多种排序方法，典型：
    - 直接插入排序
    - 折半插入排序
    - 希尔(Shell)排序

基本排序方法

- **直接插入排序**
  - 当插入第i个元素时，前$i-1$个元素已经排好序，用第i个元素关键字和前$i-1$个元素关键字顺序比较，确定插入位置

```
void insert_sort(int *a, int left, int right){
    int i,j, tTag;
    for(i=right; i>left; i--)          //最小元素置于首位
    {if(a[i-1]>a[i]; exch(a[i-1], a[i]);}
    for(i=left+2; i<=right; i++){
        j=i;    tTag=a[i];             //保存待插入元素
        while(tTag<a[j-1]){            //不是插入位置
            a[j]=a[j-1];    j--;       //元素后移
        }
        a[j]=tTag;                     //插入位置
    }
}
```

基本排序方法

	65	83	79	82	84	73	78	71	69	88	65	77	80	76	69
1	65	**83**	79	82	84	73	78	71	69	88	65	77	80	76	69
2	65	**79**	83	82	84	73	78	71	69	88	65	77	80	76	69
3	65	79	**82**	83	84	73	78	71	69	88	65	77	80	76	69
4	65	79	82	83	**84**	73	78	71	69	88	65	77	80	76	69
5	65	**73**	79	82	83	84	78	71	69	88	65	77	80	76	69
6	65	73	**78**	79	82	83	84	71	69	88	65	77	80	76	69
7	65	**71**	73	78	79	82	83	84	69	88	65	77	80	76	69
8	65	**69**	71	73	78	79	82	83	84	88	65	77	80	76	69
9	65	69	71	73	78	79	82	83	84	**88**	65	77	80	76	69
10	65	**65**	69	71	73	78	79	82	83	84	88	77	80	76	69
11	65	65	69	71	73	**77**	78	79	82	83	84	88	80	76	69
12	65	65	69	71	73	77	78	79	**80**	82	83	84	88	76	69
13	65	65	69	71	73	**76**	77	78	79	80	82	83	84	88	69
14	65	65	69	**69**	71	73	76	77	78	79	80	82	83	84	88
	65	65	69	**69**	71	73	76	77	78	79	80	82	83	84	88

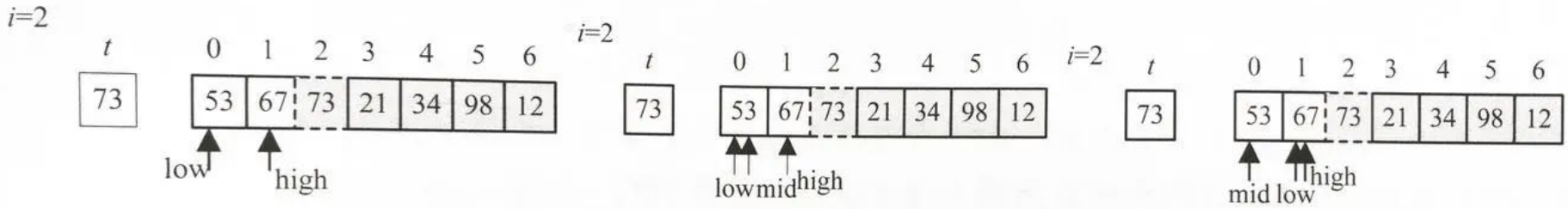
基本排序方法

- **直接插入排序的性能**
  - 需要执行$n-1$趟，关键字比较和元素移动次数与初始序列相关
  - 若初始序列已经有序，可以做到总比较次数$n-1$，元素移动次数$2(n-1)$
  - 若初始序列是反序的，则总的比较次数和元素移动次数均为$n(n-1)/2$
  - 平均情况下总的比较次数和元素移动次数为$n^2/2$
  - 直接插入排序是稳定的排序算法，时间复杂度为$O(n^2)$

基本排序方法

- **折半插入排序**
  - 当插入第i个元素时，前$i-1$个元素已经排好序，利用折半查找法确定插入位置

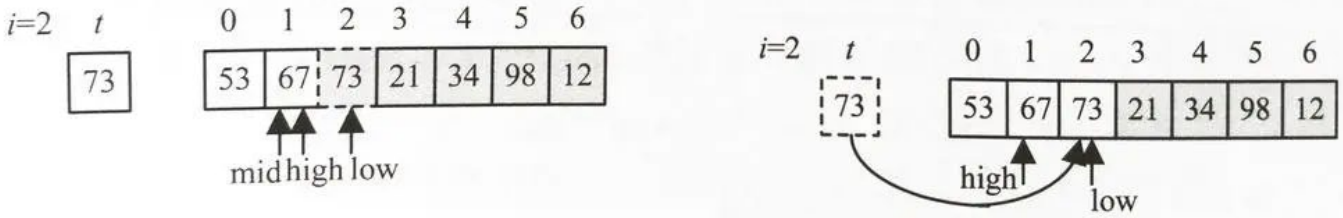
```
void insert_bisort(int *a, int left, int right){
   int i,j,r,l,middle,e;
   for(i=left+1; i<=right; i++){                //折半查找
     e=a[i];  l=left;  r=i-1;
     while(l<=r){
       middle=l+(r-l)/2;
       if(e<a[middle])   r=middle-1;
       else   l=middle+1;
     }
     for(j=i-1; j>=l; j--)  a[j+1]=a[j];   //空出插入位置
     a[l]=e;                               //元素插入
   }
}
```



（a）初始时有low=0，high=1

（b）因为low=0，high=1，所以有mid=(low+high)/2=0

（c）比较a[mid]与t的大小，因为73> 53 所以将low加1，故low=1



（d）因为low=high=1，所以mid=1。比较a[mid]与*t*的大小，因为73>67，所以将low加1，故low=2

（e）因为low>i-1，所以不需要移动元素，直接将73插入a[low]中

```
void insert_bisort(int *a, int left, int right){
   int i,j,r,l,middle,e;
   for(i=left+1; i<=right; i++){              //折半查找
     e=a[i];  l=left;  r=i-1;
     while(l<=r){
       middle=l+(r-l)/2;
       if(e<a[middle])   r=middle-1;
       else   l=middle+1;
     }
     for(j=i-1; j>=l; j--)  a[j+1]=a[j];   //空出插入位置
     a[l]=e;                                //元素插入
   }
}
```

基本排序方法

- **折半插入排序的性能**
  - 关键字比较次数与初始序列无关
  - 插入第i个元素时，需要经过$\lfloor \log_2(i-1) \rfloor + 1$次比较
  - 总的比较次数为$O(n \log n)$
  - 比较次数较直接插入平均/最差情况好，**较最好情况差**
  - 数据元素的移动次数与直接插入排序相同
  - 总的时间复杂度为$O(n^2)$

基本排序方法

- **希尔排序(缩小增量排序)**
  - $\boldsymbol{h}$-排序序列：序列中间距为h的元素都有序
  - 设计递增步长序列$h_1, h_2, \ldots, h_k$，其中$h_1 = 1$，反向使用该序列指导排序过程：
    - 第一步将序列变成h_k – 排序序列
    - 第二步将序列变成h_{k-1} – 排序序列
    - 如此继续直到进行完k步，排序即完成
  - 排序过程中每一步，按照元素间隔h_i将序列分为若干个子序列，分别实施直接插入排序
  - 间隔较大时，子序列元素少，排序速度快
  - **间隔较小时，前面大部分元素已经有序，因此排序速度依然较快**

步长序列：13,4,1

13-排序

1	65	83	79	82	84	73	78	71	69	88	65	77	80	76	69
2	**65**	83	79	82	84	73	78	71	69	88	65	77	80	**76**	69
3	65	**69**	79	82	84	73	78	71	69	88	65	77	80	76	**83**

4-排序

1	**65**	69	79	82	**84**	73	78	71	69	88	65	77	80	76	83
2	65	**69**	79	82	84	**73**	78	71	69	88	65	77	80	76	83
3	65	69	**78**	82	84	73	**79**	71	69	88	65	77	80	76	83
4	65	69	78	**71**	84	73	79	**82**	69	88	65	77	80	76	83
5	**65**	69	78	71	**69**	73	79	82	**84**	88	65	77	80	76	83
6	65	**69**	78	71	69	**73**	79	82	84	**88**	65	77	80	76	83
7	65	69	**65**	71	69	73	**78**	82	84	88	**79**	77	80	76	83
8	65	69	65	**71**	69	73	78	**77**	84	88	79	**82**	80	76	83
9	**65**	69	65	71	**69**	73	78	77	**80**	88	79	82	**84**	76	83
10	65	**69**	65	71	69	**73**	78	77	80	**76**	79	82	84	**88**	83
11	65	69	**65**	71	69	73	**78**	77	80	76	**79**	82	84	88	**83**

1-排序

1	65	69	65	71	69	73	78	77	80	76	79	82	84	88	83
2	65	**65**	**69**	71	69	73	78	77	80	76	79	82	84	88	83
3	65	65	69	71	69	73	78	77	80	76	79	82	84	88	83
4	65	65	69	**69**	**71**	73	78	77	80	76	79	82	84	88	83
5	65	65	69	69	71	73	78	77	80	76	79	82	84	88	83
6	65	65	69	69	71	73	78	77	80	76	79	82	84	88	83
7	65	65	69	69	71	73	**77**	**78**	80	76	79	82	84	88	83
8	65	65	69	69	71	73	77	78	80	76	79	82	84	88	83
9	65	65	69	69	71	73	**76**	**77**	**78**	**80**	79	82	84	88	83
10	65	65	69	69	71	73	76	77	78	**79**	**80**	82	84	88	83
11	65	65	69	69	71	73	76	77	78	79	80	82	84	88	83
12	65	65	69	69	71	73	76	77	78	79	80	82	84	88	83
13	65	65	69	69	71	73	76	77	78	79	80	82	84	88	83
14	65	65	69	69	71	73	76	77	78	79	80	82	84	88	83
	65	65	69	69	71	73	76	77	78	79	80	82	84	88	83

基本排序方法

- **希尔排序的性能：取决于步长的选择**
  - Shell最初提出(1959年)$h_{i-1} = \lfloor h_i/2 \rfloor$，直到最后一步奇数位置元素才和偶数位置元素比较，效率低
  - Knuth提出$h_{i-1} = \lfloor h_i/3 \rfloor + 1$，将效率大幅提高
  - 步长的选择方法：
    - 多种观点：取奇数，步长值互质，…
    - 一个较好的步长序列：$h_i = 4^i + 3 \times 2^{i-1} + 1$ (1,8,23,77,…)
  - 复杂度与步长序列的关系尚无完整的结论
  - Knuth认为当n很大时，大约在$n^{1.25} \sim 1.6n^{1.25}$
  - 效率高，编码简单易执行，应用广泛

基本排序方法— 希尔排序

```
void Shell_Sort(int *a,int n)
{
    int i,j,h,tmp;
    for(h=n/2; h>0 ;h/=2)
    {
        for(i=h;i<n;++i)
        {
            tmp=a[i];
             for(j=i; j>=h&&a[j-h]>tmp; j-=h) //与前面相隔h的元素比较
                 a[j]=a[j-h];  //后移
             a[j]=tmp;
        }
    }
}
```

基本排序方法

- **直接选择排序**
  - 在数据元素中选择最小关键字元素，将其与首元素交换；在剩余的序列中重复执行直到仅剩一个元素
  - 总的关键字比较次数为$n(n-1)/2$，交换次数为$n-1$
  - 实现简单执行时间固定，适用于元素规模大但关键项比较小的情况；不稳定的排序

```
void select_sort(int *a, int left, int right){
    int i,j,min;
    for(i=left; i<right; i++){
        min = i;
        for(j=i+1; j<=right; j++)      //选择序列的最小元素
            if(a[j]<a[min])  min=j;
        exch(a[i], a[min]);            //元素交换
    }
}
```

	65	83	79	82	84	73	78	71	69	88	65	77	80	76	69
1	65	83	79	82	84	73	78	71	69	88	**65**	77	80	76	69
2	65	65	79	82	84	73	78	71	**69**	88	83	77	80	76	69
3	65	65	69	82	84	73	78	71	79	88	83	77	80	76	**69**
4	65	65	69	69	84	73	78	**71**	79	88	83	77	80	76	82
5	65	65	69	69	71	**73**	78	84	79	88	83	77	80	76	82
6	65	65	69	69	71	73	78	84	79	88	83	77	80	**76**	82
7	65	65	69	69	71	73	76	84	79	88	83	**77**	80	78	82
8	65	65	69	69	71	73	76	77	79	88	83	84	80	**78**	82
9	65	65	69	69	71	73	76	77	78	88	83	84	80	**79**	82
10	65	65	69	69	71	73	76	77	78	79	83	84	**80**	88	82
11	65	65	69	69	71	73	76	77	78	79	80	84	83	88	**82**
12	65	65	69	69	71	73	76	77	78	79	80	82	**83**	88	84
13	65	65	69	69	71	73	76	77	78	79	80	82	83	88	84
14	65	65	69	69	71	73	76	77	78	79	80	82	83	84	88
	65	65	69	69	71	73	76	77	78	79	80	82	83	84	88

基本排序性能比较

排序方法	比较次数		移动次数		稳定性
	最好情况	最坏情况	最好情况	最坏情况	
直接插入排序	$O(n)$	$O(n^2)$	$O(n)$	$O(n^2)$	Y
折半插入排序	$O(nlogn)$		$O(n)$	$O(n^2)$	Y
冒泡排序	$O(n)$	$O(n^2)$	0	$O(n^2)$	Y
选择排序	$O(n^2)$		$O(n)$		N

- 基本排序方法的平均复杂时间度均是$O(n^2)$
- 平均情况下：折半插入排序好于直接插入排序；直接插入排序好于冒泡排序
- 直接插入排序和改进的冒泡排序最好情况下性能好
- 直接选择排序适用于数据大而关键字小的情况
- 希尔排序的复杂度尚不明晰，实际中适用于中等规模序列

本节内容

- 排序的概念
- 基本排序方法
- 快速排序
- 归并排序
- 堆排序
- 线性复杂度排序

快速排序

- 快速排序和冒泡排序都属于交换类的排序
- 最早由Hoare在莫斯科州立大学访问时提出(1960)
- 性能优良使用广泛，已经成为标准方法(qsort)
- 快速排序的基本思想：
  - 选取序列中某元素$R_{[i]}$
  - 将序列划分(partition)成左右两个子序列，左子序列元素关键均小于$R_{[i]}$，右子序列元素关键均大于$R_{[i]}$
  - 对左右子序列重复实施上述划分操作
  - 每次划分使得一个元素放置到正确位置上，快速排序可以采用递归实现

快速排序

```
void quick_sort(int *a, int left, int right){
    if(right<=left) return;
    int pos = partition(a, left, right);
    quick_sort(a, left, pos-1);
    quick_sort(a, pos+1, right);
}
```

快速排序

- **划分操作(partition)**
  - l、r分别指向首、尾元素；i、j分别为左、右指针
  - 假设选择尾（头）元素v为划分元素
  - i自左向右扫描，找到一个大于v的元素
  - j自右向左扫描，找到一个小于v的元素
  - 交换这两个元素
  - 继续扫描，直到i、j交错，将i处的元素与v交换， v的排序位置就在i

相对于v，逆序

快速排序

划分的实现

```
int partition(int *a, int left, int right){
    int i=left-1; j=right;
    int e = a[right];                          //划分元素
    while(1){
        while(a[++i]<=e) if(i==right) break;   //向右寻找大于e的
        while(a[--j]>=e) if(j==left)  break;   //向左寻找小于e的
        if(i>=j) break;                        //划分位置
        exch(a[i], a[j]);
    }
    exch(a[i], a[right]);                      //划分元素就位
    return i;
}
```

一次划分

65	83	79	82	84	73	78	71	69	88	65	77	80	76	69
65	**83**	79	82	84	73	78	71	69	88	**65**	77	80	76	69
65	**65**	79	82	84	73	78	71	69	88	**83**	77	80	76	69
65	65	**79**	82	84	73	78	71	69	88	83	77	80	76	69
65	65	**69**	82	84	73	78	71	69	88	83	77	80	76	79

左子序列　　右子序列

快速排序

65	83	79	82	84	73	78	71	69	88	65	77	80	76	69
65	65	**69**	82	84	73	78	71	69	88	83	77	80	76	79
65	**65**		76	77	73	78	71	69	**79**	88	83	84	80	82
65			**69**	77	73	78	71	76		80	**82**	83	80	88
				71	73	**76**	77	78		80		83	84	88
				71	**73**		77	**78**		**80**		83	**84**	
				71			**77**					**83**		
65	**65**	**69**	**69**	**71**	**73**	**76**	**77**	**78**	**79**	**80**	**82**	**83**	**84**	**88**

快速排序

- **快速排序的性能**
  - 最好情况：每次划分左右子序列等长，递归树为AVL树，此时的时间复杂度为$O(n\log_2 n)$
  - 平均情况的时间复杂度也是$O(\mathrm{n}\log_2 n)$，但以常数倍略差于最好情况(约2倍)，好于基本排序方法
  - 最坏情况：序列有序，每次划分只能得到一侧的子序列，递归树退化为单枝树，时间复杂度退化为$O(n^2)$
  - 空间复杂度：递归栈平均情况下$O(\log_2 n)$，最坏情况为$O(n)$
  - 不稳定的排序方法

快速排序

- **快速排序的改进**
  - 当序列很小的时候，容易产生子序列不平衡而接近最坏情况，应避免继续递归调用：
    - 当子序列规模小于门限，直接对其采用插入排序
    - 当子序列足够小，不排序而直接跳过；完成快排之后再调用直接插入排序，此时序列基本有序，接近最好情况
  - 为规避最坏情况的发生，应合理选择划分元素:
    - 随机选择划分元素
    - 随机选择三个元素，以其**中值**作为划分元素

快速排序

改进的快速排序

```
void quick_sort(int *a, int left, int right){
    if(right-left<=M) return;                    //序列过短则跳过
    exch(a[(left+right)/2], a[right-1]); //三元素中值
    compExch(a[left], a[right-1]);
    compExch(a[left], a[right]);
    compExch(a[right-1], a[right]);
    int pos = partition(a, left, right);
    quick_sort(a, left, pos-1);
    quick_sort(a, pos+1, right);
}

void hybrid_sort(int *a, int left, int right){
    quick_sort(a, left, right);
    insert_sort(a, left, right);
}
```

本节内容

- 排序的概念
- 基本排序方法
- 快速排序
- 归并排序
- 堆排序
- 线性复杂度排序

归并排序

- **二路归并**
  - 归并：将两个或多个序列合并成一个新的序列
  - 二路归并：将两个有序序列保序归并

```
void merge(int *c, int *a, int n, int *b, int m){
    int i,j,k;
    for(i=0,j=0,k=0; k<n+m; k++){
        if(i==n) {c[k]=b[j]; j++; continue;}
        if(j==m) {c[k]=a[i]; i++; continue;}
        c[k]=(a[i]<b[j])? a[i++]:b[j++];
    }
}
```

  - 归并后序列长度为n，二路归并时间复杂度$O(n)$，空间复杂度$O(n)$.

归并排序

- **自顶向下的归并排序**
  - 类似快速排序，自顶向下不断缩小序列规模
  - 将序列分成两个大致等长的左右子序列
  - 对两个子序列分别递归地进行排序
  - 排序性能与自底向上归并排序一致

```
void merge_sortA(int *a, int left, int right){
    if(right<=left) return;
    int m=left+(right-left)/2;
    merge_sortA(a, left, m);
    merge_sortA(a, m+1, right);
    merge(a, left, m+1, right);
}
```

	65	83	79	82	84	73	78	71	69	88	65	77	80	76	69
1	65	83													
1			79	82											
2	65	79	82	83											
1					73	84									
1							71	78							
2					71	73	78	84							
3	65	71	73	78	79	82	83	84							
1									69	88					
1											65	77			
2									65	69	77	88			
1													76	80	
1															69
2													69	76	80
3									65	69	69	76	77	80	88
4	65	65	69	69	71	73	76	77	78	79	80	82	83	84	88
	65	65	69	69	71	73	76	77	78	79	80	82	83	84	88

递归分解

回退归并

归并排序

- **自底向上的归并排序**
  - 将序列看成n个长度为1的子序列
  - 两两二路归并得到$O(\left\lceil\frac{n}{2}\right\rceil)$个长度为2的有序子序列
  - 重复做两两二路归并，直到得到有序序列

```
//将a[p1]~a[p2-p1]与a[p2]~a[p3]二路归并存放于a[p1]~a[p3]
mergeA(int *a, int p1, int p2, int p3);
void merge_sort(int *a, int left, int right){
    int i,m;
    for(m=1; m<=right-left; m+=m){
        for(i=left; i<=right-m; i+=m+m)
            mergeA(a,i,i+m,min(i+m+m-1, right));
    }
}
```

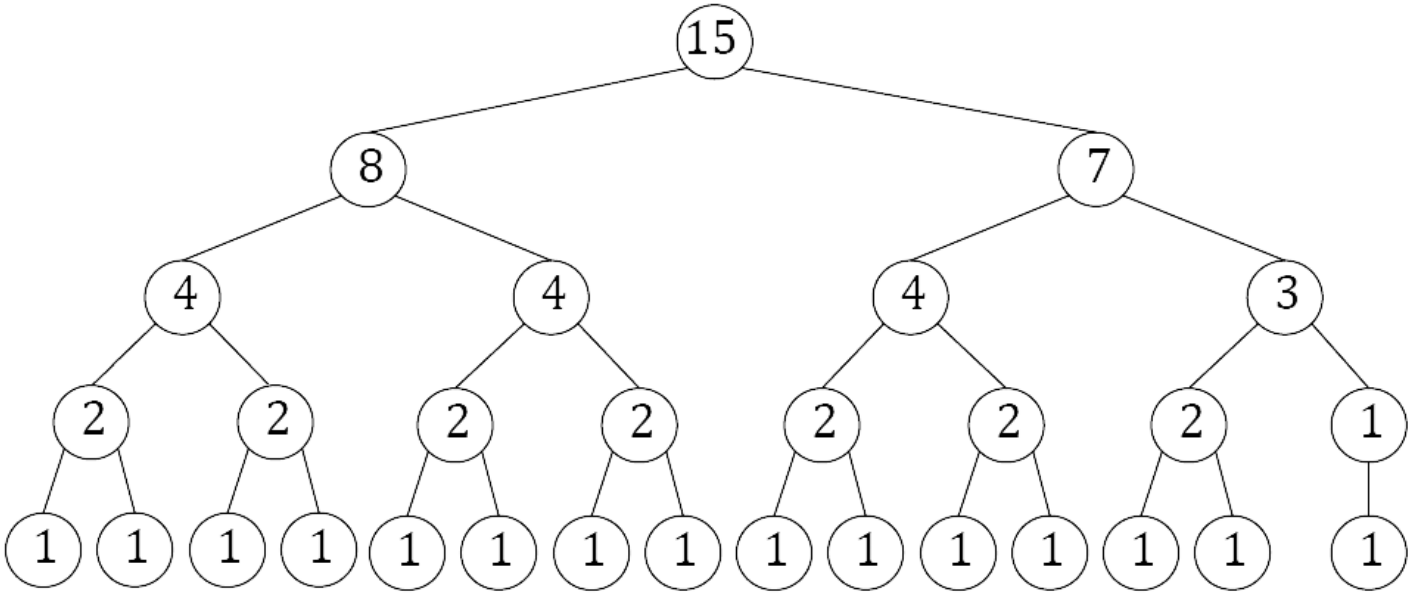
归并排序

自底向上的归并排序

	65	83	79	82	84	73	78	71	69	88	65	77	80	76	69
1	65	83	79	82	73	84	71	78	69	88	65	77	76	80	69
2	65	79	82	83	71	73	78	84	65	69	77	88	69	76	80
3	65	71	73	78	79	82	83	84	65	69	69	76	77	80	88
	65	65	69	69	71	73	76	77	78	79	80	82	83	84	88

从叶子开始

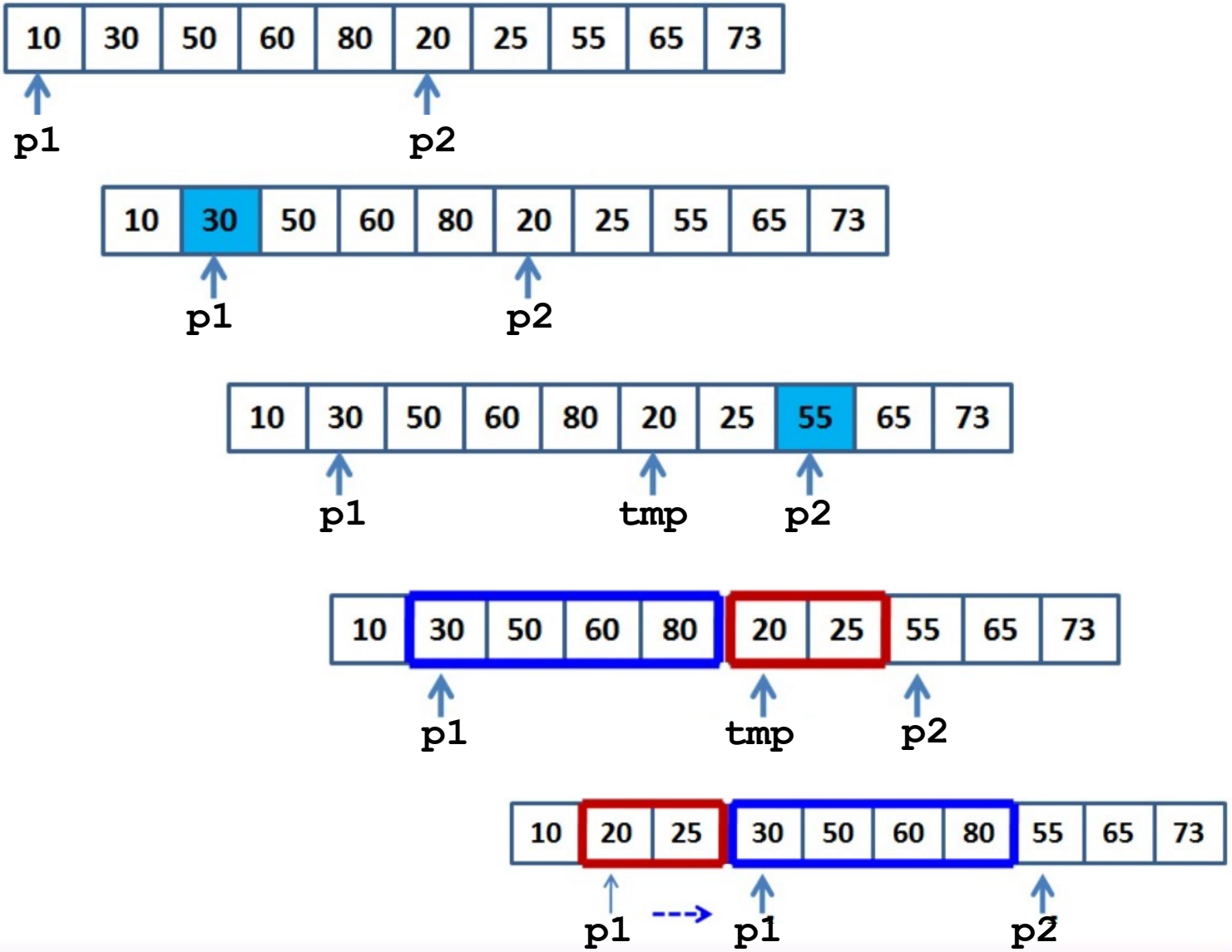
归并树



归并排序

- **自底向上归并排序的性能**
  - 序列长度为n，每趟归并的时间复杂度为$O(n)$
  - 归并的总趟数为$\lceil \log_2 n \rceil$
  - 总的时间复杂度为$O(n \log_2 n)$
  - **性能与初始序列无关**，不会出现性能恶化
  - 需要占用$O(n)$的额外存储空间
  - 如果**归并是稳定**的，归并排序也是稳定的
  - 只涉及顺序的数据访问，适用于链表排序

原地（in-place）归并排序



归并排序

- 不存在某种排序算法对所有的应用条件都适用
- 应根据实际任务的特点适当选择合适的排序算法
- 可以把不同的排序算法混合使用，发挥各自的优点

本节内容

- 排序的概念
- 基本排序方法
- 快速排序
- 归并排序
- 堆排序
- 线性复杂度排序

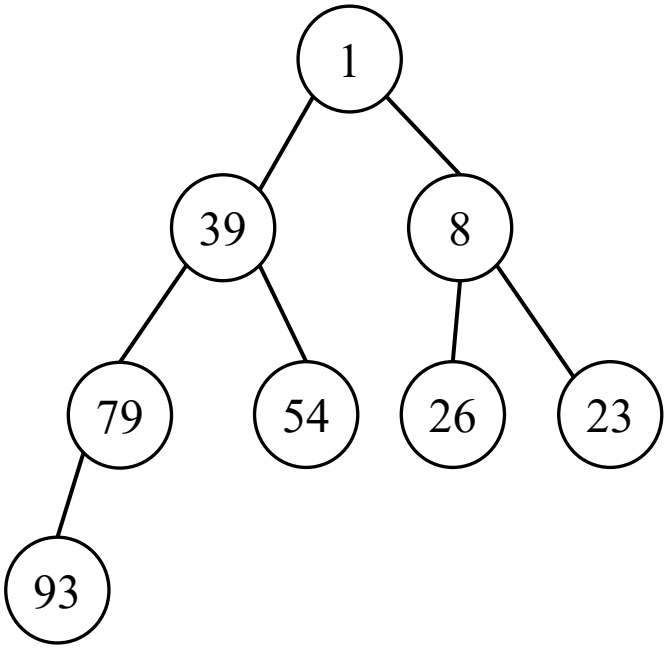
二叉树的应用

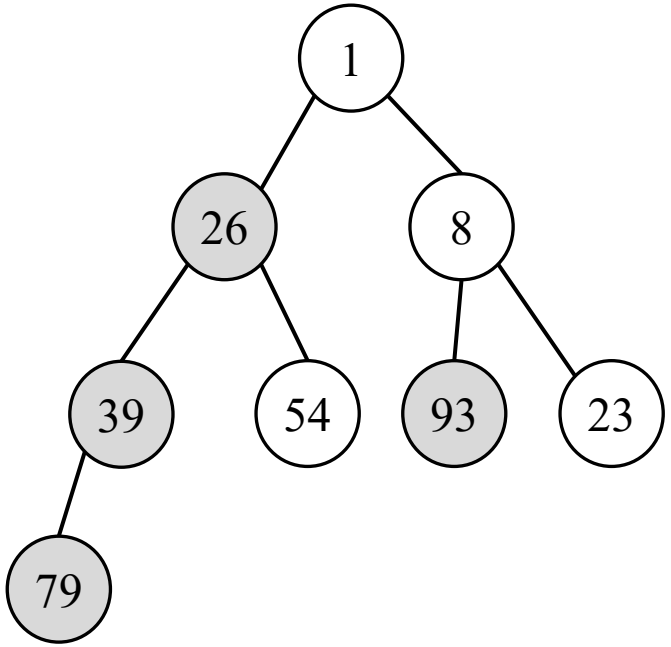
- **堆(Heap)**
  - 完全二叉树
  - 最小堆：子节点的值总是大于根节点
  - 最大堆：子节点的值总是小于根节点
  - 和计算机存储空间“堆”没有关系
  - 适合用来实现优先队列 (priority queue)

堆：满二叉树

- 和二叉搜索树一样，一个序列对应的堆不唯一

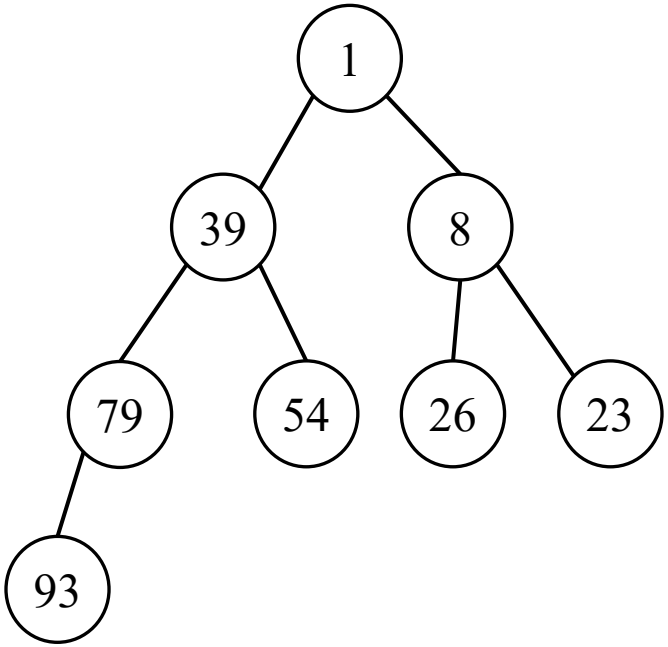
{1, 8, 23, 26, 39, 54, 79, 93}





堆

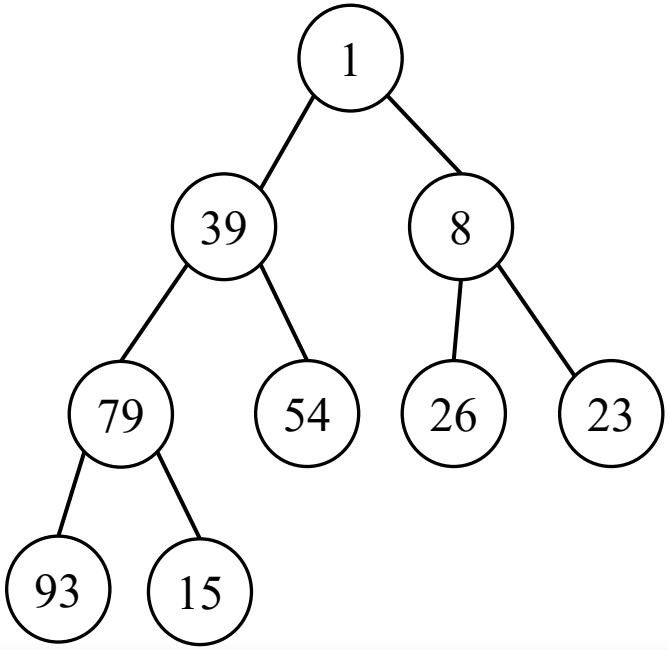
- **堆的插入操作**



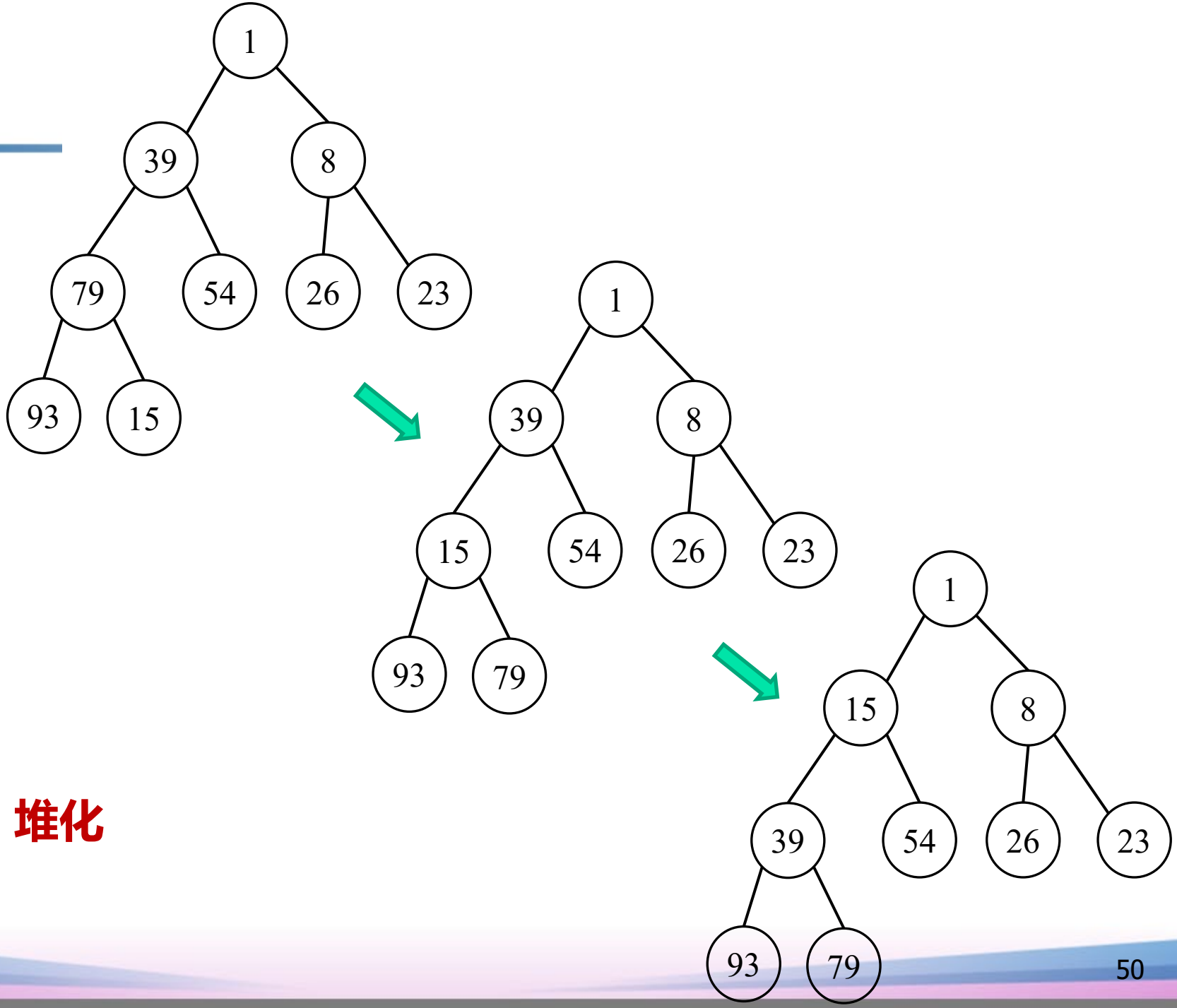
插入15?

堆

- **堆的插入操作**
  - 首先把待插入节点放为二叉树最下层最右侧叶节点
  - 但破坏了堆的定义，需要进行堆化（和父节点对换）



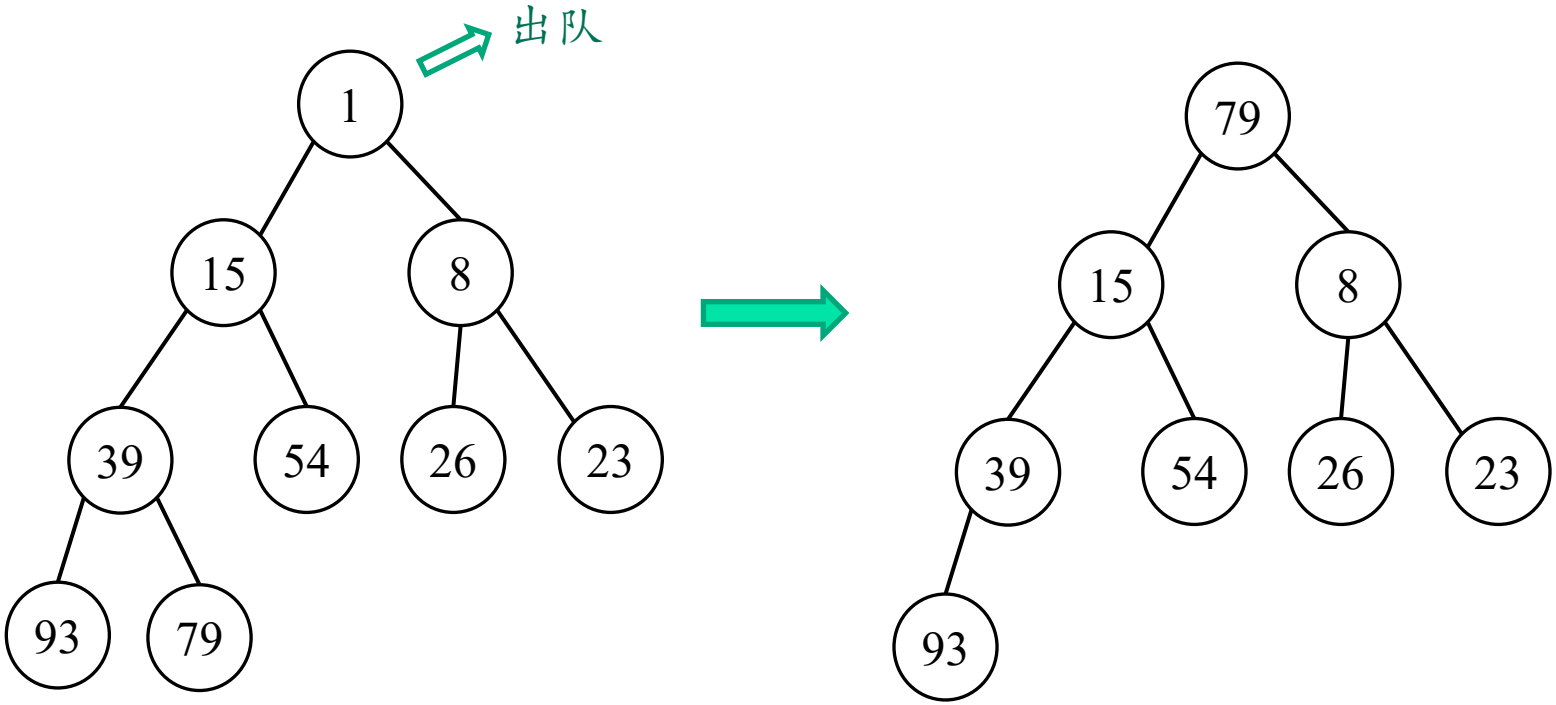
堆

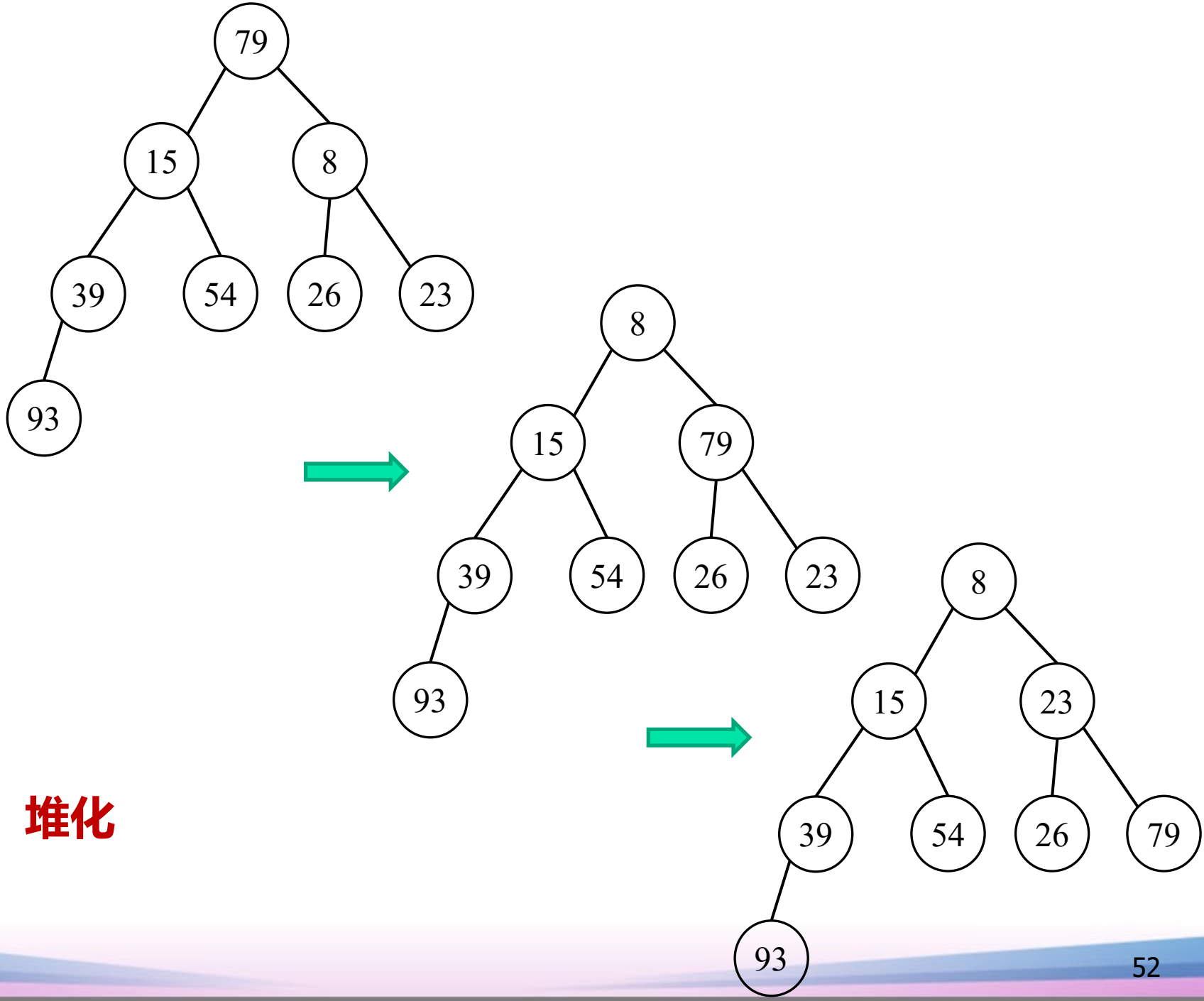


堆化

堆

- **删除堆顶操作(优先先元素出队)**
  - 首先找到最底层最右侧节点，用该节点替代根节点并删除其本身

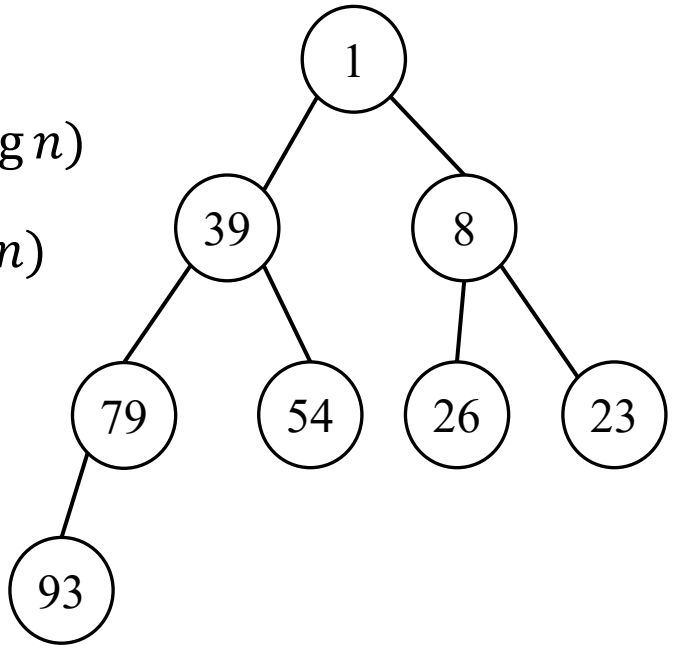




堆化

堆

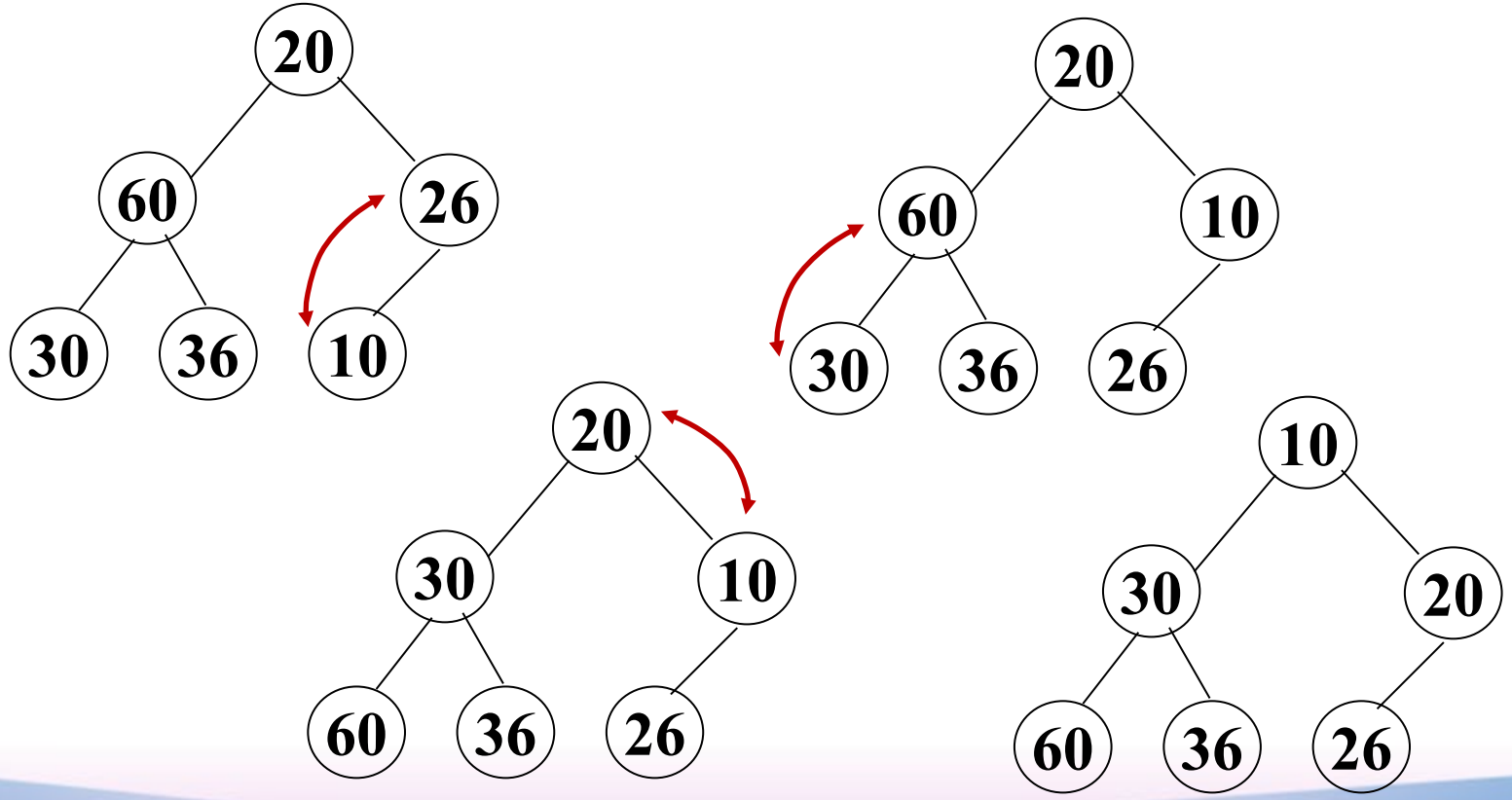
- **堆的性质**
  - n个节点的堆的高度为$O(\log n)$
  - 插入/删除堆顶操作的时间复杂度均为$O(\log n)$
  - 给定序列创建一个堆时间复杂度可以为$O(n)$
  - 可以用于实现堆排序(heap sort)
  - 堆可以用完全二叉树的顺序存储方式实现
  - 结点i的父结点位置$\frac{i-1}{2}$
  - 结点i的左孩子为$2i+1$，右孩子为$2i+2$



1	39	8	79	54	26	23	93							

堆排序：自底向上构造堆

- 将序列{ 20, 60, 26, 30, 36, 10 }构建小根堆
- 从非叶子结点开始考察，即$\lfloor n/2 \rfloor - 1$， ……, 0 自右向左，自下向上，反层序遍历非叶子结点，逐步调整



堆排序

- **自底向上建堆的复杂度：**

变量替换

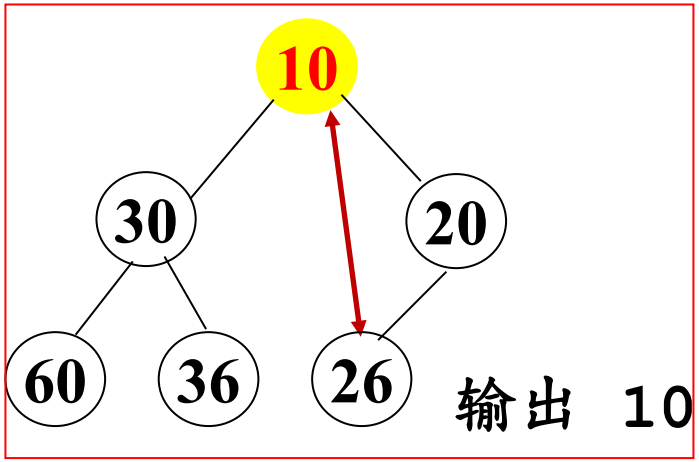
$$T=\sum_{k=1}^{h}2^{k-1}O(h-k+1)=\sum_{k=1}^{h}2^{h-k}O(k)=O\left(\sum_{k=1}^{h}k2^{h-k}\right)$$

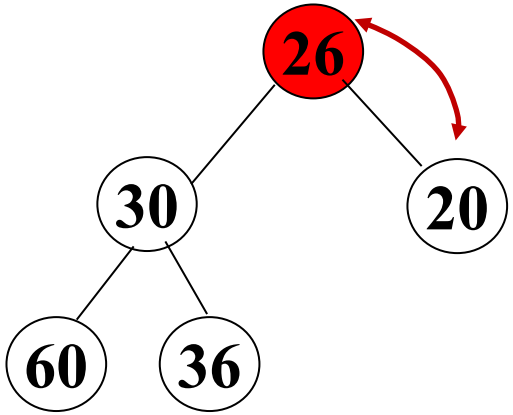
i层至多2^{i-1}个结点

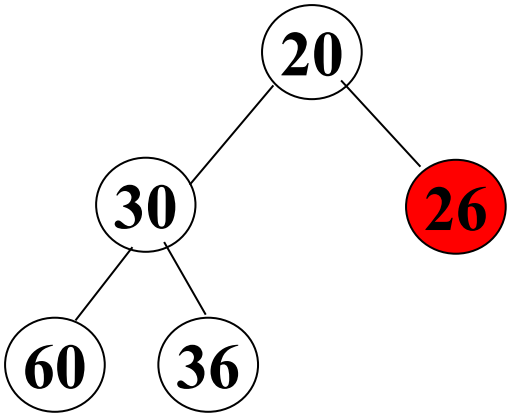
$$=O\left(2^{h}\sum_{k=1}^{h}k/2^{k}\right)=O\left(2^{h+1}-h-2\right)=O(n)$$

第i层自底向上堆化的复杂度：$O(h-i+1)$

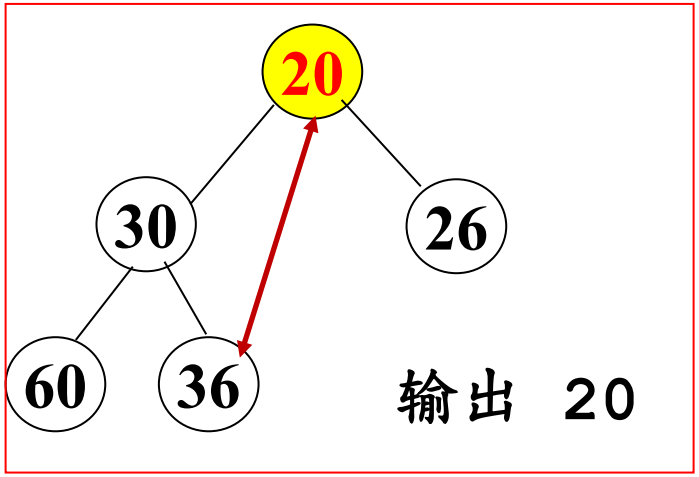
堆排序

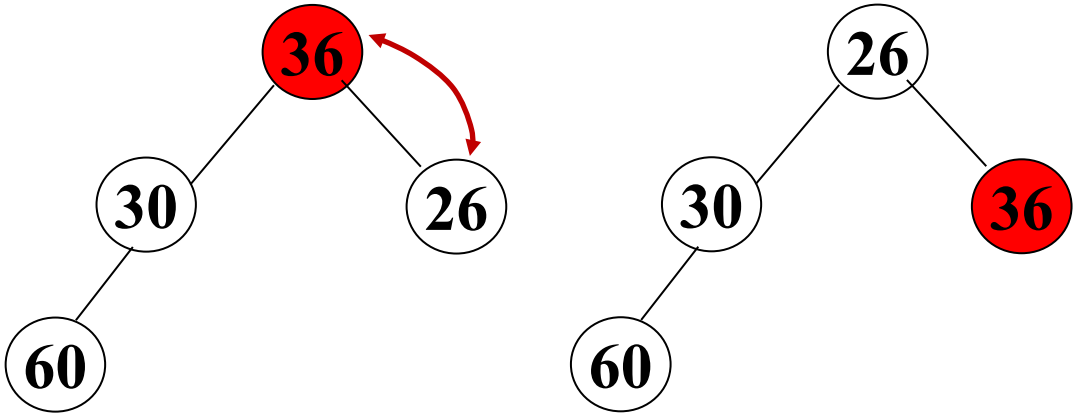




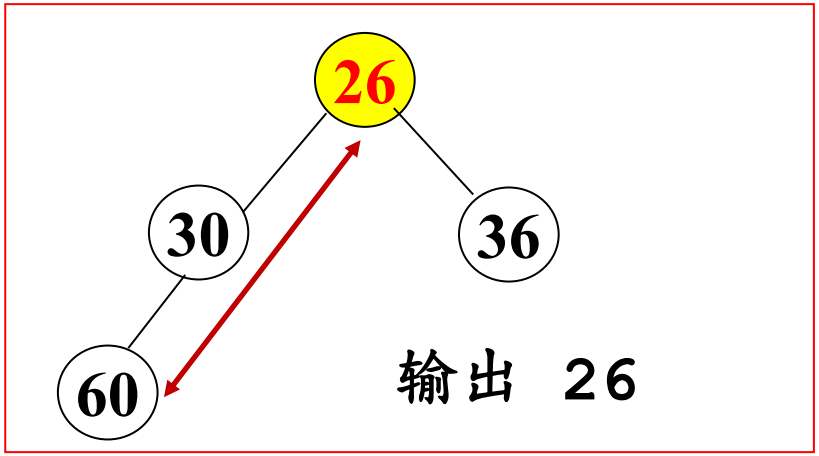


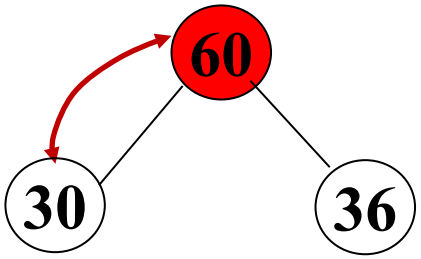
堆排序

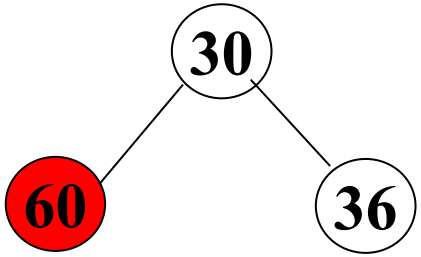




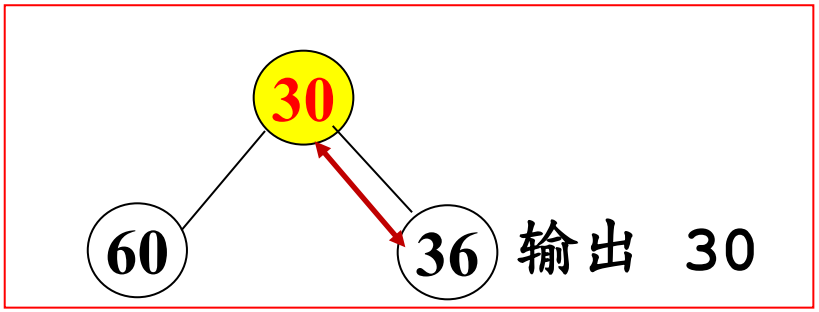
堆排序

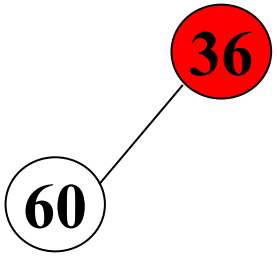




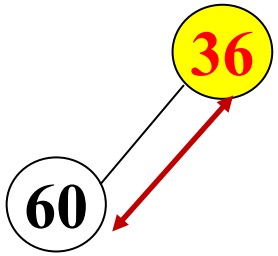


堆排序





堆排序



输出 36

60

输出 60

堆排序

- **堆排序(Heap Sort)**
  - 自底向上建堆复杂度 $O(n)$
  - 依次输出树根并堆化 $O(nlogn)$

本节内容

- 排序的概念
- 基本排序方法
- 快速排序
- 归并排序
- 堆排序
- 线性复杂度排序

线性复杂度排序

- **计数排序**
  - 一般的，排序时间复杂度的下限是$O(n\log n)$（why?）
  - 一些特殊序列，**可利用其特性实现线性复杂度排序**
  - 关键字为整数，取值范围$[0,k]$，可通过关键字取值的累计计数直接计算出各元素在有序序列中的位置

A	2	5	3	0	2	3	0	3

待排序序列

取值范围[0,5]

	0	1	2	3	4	5
C_0	2	0	2	3	0	1

关键字计数

	0	1	2	3	4	5
C	2	2	4	7	7	8

关键字累计计数

线性复杂度排序

- **计数排序**

A	2	5	3	0	2	3	0	3

  - 输出序列B，用来存放排序的结果
  - 从$i=7$，即序列最后一个元素开始，反向执行：

$$B[C[A[i]]-1]=A[i],\ C[A[i]]=C[A[i]]-1,\ i=i-1$$

	0	1	2	3	4	5	6	7
$i=7$							**3**	
$i=6$		**0**					3	
$i=5$		0				**3**	3	
$i=4$		0		**2**		3	3	
$i=3$	**0**	0		2		3	3	
$i=2$	0	0		2	**3**	3	3	
$i=1$	0	0		2	3	3	3	**5**
$i=0$	0	0	**2**	2	3	3	3	5

有序序列B的生成过程

	0	1	2	3	4	5
	2	2	4	**6**	7	8
	1	2	4	6	7	8
	1	2	4	**5**	7	8
	1	2	**3**	5	7	8
	0	2	3	5	7	8
	0	2	3	**4**	7	8
	0	2	3	4	7	**7**
	0	2	**2**	4	7	7

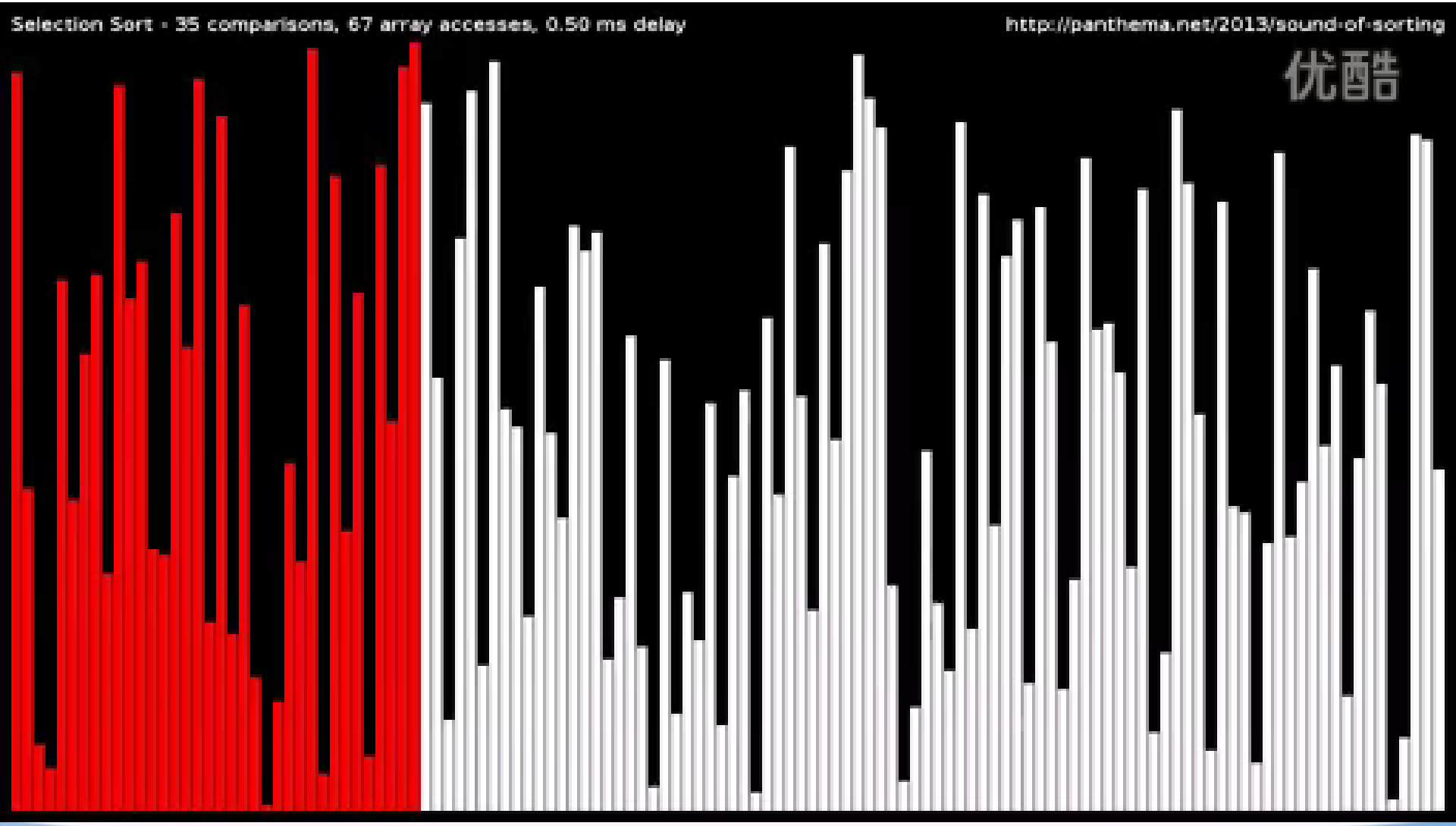
C的变化过程

C	0	1	2	3	4	5
	2	2	4	7	7	8

性能比较

排序方法	比较次数		移动次数		稳定性	附加存储	
	最好	最差	最好	最差		最好	最差
直接插入	$O(n)$	$O(n^2)$	$O(n)/0$	$O(n^2)$	Y	$O(1)$	
折半插入	$O(n\log_2 n)$		$O(n)$	$O(n^2)$	Y	$O(1)$	
冒泡	$O(n)$	$O(n^2)$	0	$O(n^2)$	Y	$O(1)$	
选择	$O(n^2)$		$O(n)$		N	$O(1)$	
快速	$O(n\log_2 n)$	$O(n^2)$	$O(n\log_2 n)$	$O(n^2)$	N	$O(\log_2 n)$	$O(n)$
归并	$O(n\log_2 n)$ 引入了一个与初始序列长度相同的数组来存储合并后的结果，因而不涉及交换				Y	$O(n)$	
堆排序	$O(n\log_2 n)$		$O(n\log_2 n)$		N	$O(1)$	
计数排序	$O(n+k)$，n为输入元素个数，k为待排序列中最大的数				N	$O(k)$	

直接插入排序在程序实现上，通常会用一个临时变量(如temp）保存待插入元素，最后又将temp移动到相应位置上，因而在很多教材上第回合插入会多2次移动操作


Selection Sort - 35 comparisons, 67 array accesses, 0.50 ms delay
http://panthema.net/2013/sound-of-sorting
优酷

谢　谢！

北京科技大学
计算机与通信工程学院